

SIMATS ENGINEERING

Name: P. NANDU VARDHAN

Reg no:192425067

Course code and Name: csa0608 – Design analysis of algorithms

Question 1: Search an Element in a Sorted Array

Given a sorted array of integers and a key value, use the Binary Search (Divide and Conquer) method to determine whether the key is present in the array. If found, print its position; otherwise, print "Element not found".

Test Case 1

Input

5
10 20 30 40 50
30

Expected Output

Element found at position 3

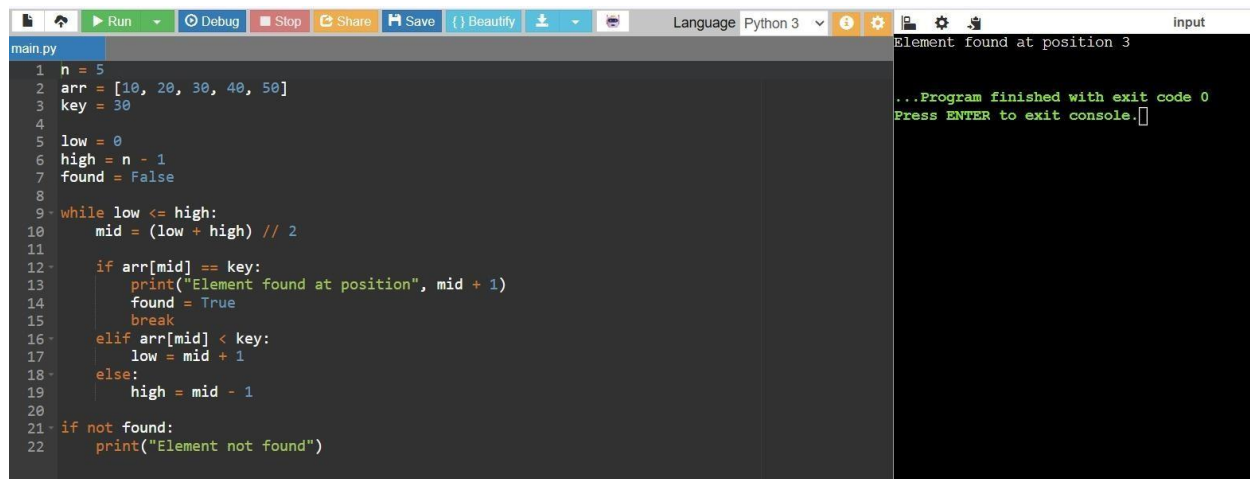
Test Case 2

Input

5
10 20 30 40 50
25

Expected Output

Element not found



The screenshot shows a Python IDE with a file named 'main.py'. The code implements a binary search algorithm. It defines a sorted array 'arr' with values [10, 20, 30, 40, 50] and a 'key' value of 30. The algorithm uses a while loop to repeatedly divide the search range (low to high) until the key is found or the range is exhausted. The output of the program is 'Element found at position 3'. The console also shows the program finished with exit code 0 and a prompt to press ENTER to exit.

```
main.py
1 n = 5
2 arr = [10, 20, 30, 40, 50]
3 key = 30
4
5 low = 0
6 high = n - 1
7 found = False
8
9 while low <= high:
10     mid = (low + high) // 2
11
12     if arr[mid] == key:
13         print("Element found at position", mid + 1)
14         found = True
15         break
16     elif arr[mid] < key:
17         low = mid + 1
18     else:
19         high = mid - 1
20
21 if not found:
22     print("Element not found")
```

Element found at position 3

...Program finished with exit code 0
Press ENTER to exit console.

Question 2: Find the Index of a Given Number

Given a sorted array and a target number, find the index of the target using Binary Search. Assume array indexing starts from 0.

Test Case 1

Input

6
2 4 6 8 10 12
8

Expected Output

Index = 3

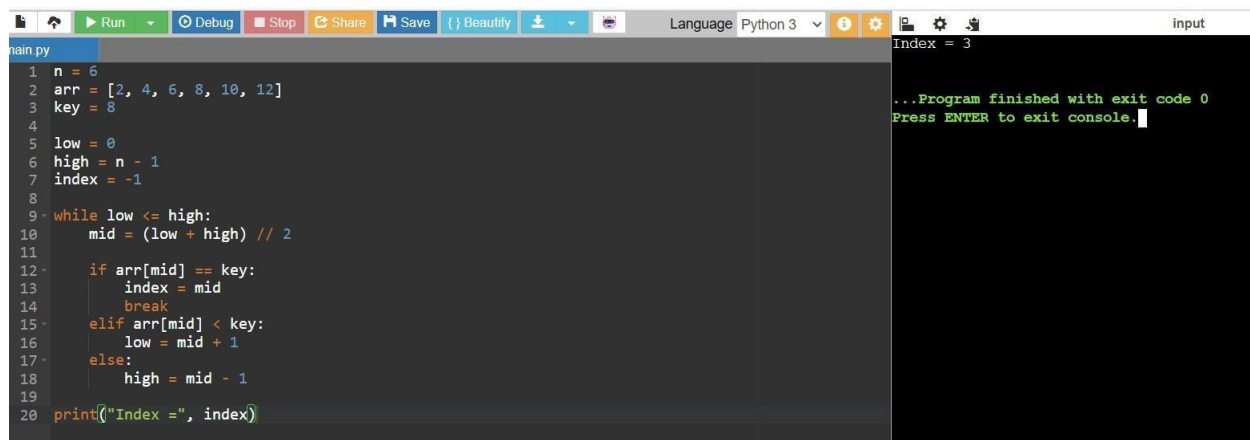
Test Case 2

Input

6
2 4 6 8 10 12
5

Expected Output

Index = -1



The screenshot shows a Python IDE with a file named 'main.py'. The code implements a binary search algorithm. It takes an array [2, 4, 6, 8, 10, 12] and a key 8. The algorithm finds the key at index 3. The output console shows 'Index = 3' and a message indicating the program finished with exit code 0.

```
1 n = 6
2 arr = [2, 4, 6, 8, 10, 12]
3 key = 8
4
5 low = 0
6 high = n - 1
7 index = -1
8
9 while low <= high:
10     mid = (low + high) // 2
11
12     if arr[mid] == key:
13         index = mid
14         break
15     elif arr[mid] < key:
16         low = mid + 1
17     else:
18         high = mid - 1
19
20 print("Index =", index)
```

Index = 3

...Program finished with exit code 0
Press ENTER to exit console.

Question 3: Count the Number of Iterations in Binary Search

Given a sorted array and a key, perform Binary Search and display the number of iterations required to find the element.

Test Case 1

Input

7
5 10 15 20 25 30 35
25

Expected Output

Element found
Iterations = 2

Test Case 2

Input

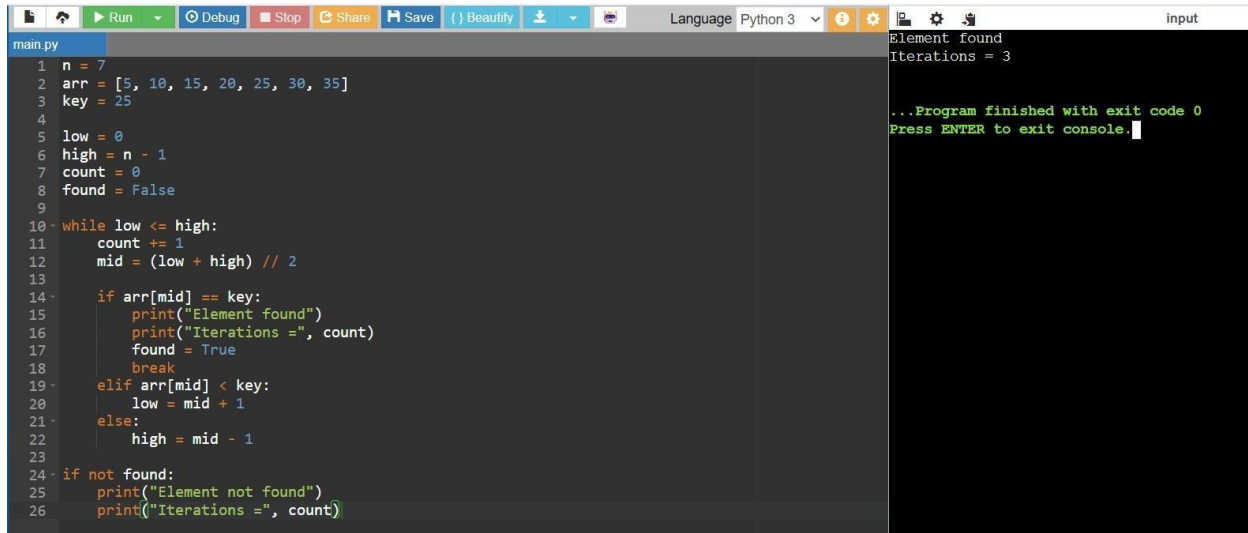
7
5 10 15 20 25 30 35

40

Expected Output

Element not found

Iterations = 3



The screenshot shows a Python IDE with a file named 'main.py'. The code implements a binary search algorithm on a sorted array [5, 10, 15, 20, 25, 30, 35] with a target key of 25. The algorithm uses a while loop to find the element, and if not found, it prints 'Element not found' and the number of iterations. The output console shows 'Element found' and 'Iterations = 3', followed by a message indicating the program finished with exit code 0.

```
1 n = 7
2 arr = [5, 10, 15, 20, 25, 30, 35]
3 key = 25
4
5 low = 0
6 high = n - 1
7 count = 0
8 found = False
9
10 while low <= high:
11     count += 1
12     mid = (low + high) // 2
13
14     if arr[mid] == key:
15         print("Element found")
16         print("Iterations =", count)
17         found = True
18         break
19     elif arr[mid] < key:
20         low = mid + 1
21     else:
22         high = mid - 1
23
24 if not found:
25     print("Element not found")
26     print("Iterations =", count)
```

Element found
Iterations = 3
...Program finished with exit code 0
Press ENTER to exit console.

Question 4: Find the First Occurrence of an Element

Given a sorted array containing duplicate elements, find the first occurrence of a given key using Binary Search.

Test Case 1

Input

8

1 2 2 2 3 4 5 6

2

Expected Output

First occurrence at index 1

Test Case 2

Input

8

1 2 2 2 3 4 5 6

7

Expected Output

Element not found

```
main.py
1 n = 6
2 arr = [2, 4, 6, 8, 10, 12]
3 key = 8
4
5 low = 0
6 high = n - 1
7 index = -1
8
9 while low <= high:
10     mid = (low + high) // 2
11
12     if arr[mid] == key:
13         index = mid
14         break
15     elif arr[mid] < key:
16         low = mid + 1
17     else:
18         high = mid - 1
19
20 print("Index =", index)
```

Index = 3

...Program finished with exit code 0
Press ENTER to exit console.

Question 5: Find the Last Occurrence of an Element

Given a sorted array containing duplicate elements, find the last occurrence of a given key using Binary Search.

Test Case 1

Input

8

1 2 2 2 3 4 5 6

2

Expected Output

Last occurrence at index 3

Test Case 2

Input

8

1 2 2 2 3 4 5 6

9

Expected Output

Element not found

```
main.py
1 n = 8
2 arr = [1, 2, 2, 2, 3, 4, 5, 6]
3 key = 2
4
5 low = 0
6 high = n - 1
7 ans = -1
8
9 while low <= high:
10     mid = (low + high) // 2
11
12     if arr[mid] == key:
13         ans = mid
14         high = mid - 1
15     elif arr[mid] < key:
16         low = mid + 1
17     else:
18         high = mid - 1
19
20 if ans != -1:
21     print("First occurrence at index", ans)
22 else:
23     print("Element not found")
```

First occurrence at index 1

...Program finished with exit code 0
Press ENTER to exit console.

Question 6: Highest and Lowest Temperature Recorded

A weather station records temperatures over a week. Find the **minimum** and **maximum** temperature using the Divide and Conquer approach.

Input

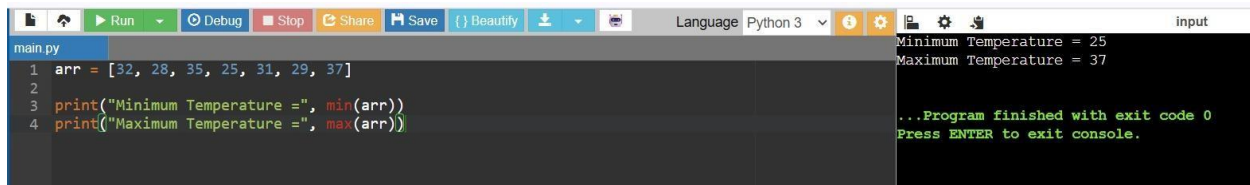
7

32 28 35 25 31 29 37

Expected Output

Minimum Temperature = 25

Maximum Temperature = 37



```
main.py
1 arr = [32, 28, 35, 25, 31, 29, 37]
2
3 print("Minimum Temperature =", min(arr))
4 print("Maximum Temperature =", max(arr))
```

Minimum Temperature = 25
Maximum Temperature = 37
...Program finished with exit code 0
Press ENTER to exit console.

Question 7: Best and Worst Student Marks

A teacher wants to identify the highest and lowest marks scored in a class using Divide and Conquer.

Input

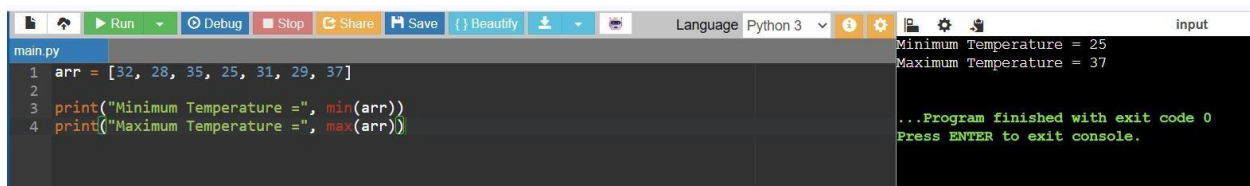
8

78 92 65 88 95 72 81 69

Expected Output

Minimum Mark = 65

Maximum Mark = 95



```
main.py
1 arr = [32, 28, 35, 25, 31, 29, 37]
2
3 print("Minimum Temperature =", min(arr))
4 print("Maximum Temperature =", max(arr))
```

Minimum Temperature = 25
Maximum Temperature = 37
...Program finished with exit code 0
Press ENTER to exit console.

Question 8: Stock Price Analysis

An investor wants to know the minimum and maximum stock prices recorded during a trading week.

Input

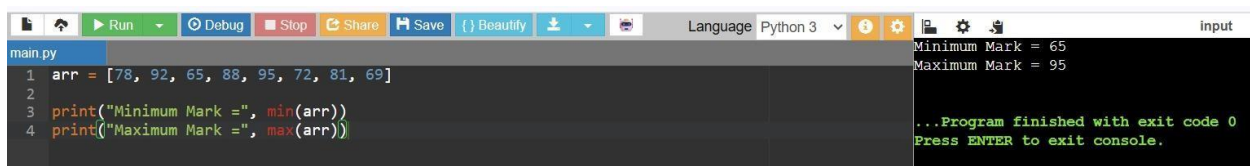
6

450 520 480 610 430 590

Expected Output

Minimum Stock Price = 430

Maximum Stock Price = 610



```
main.py
1 arr = [78, 92, 65, 88, 95, 72, 81, 69]
2
3 print("Minimum Mark =", min(arr))
4 print("Maximum Mark =", max(arr))
```

Minimum Mark = 65
Maximum Mark = 95
...Program finished with exit code 0
Press ENTER to exit console.

Question 9: Fastest and Slowest Race Times

A sports analyst needs to find the fastest (minimum) and slowest (maximum) race completion times.

Input

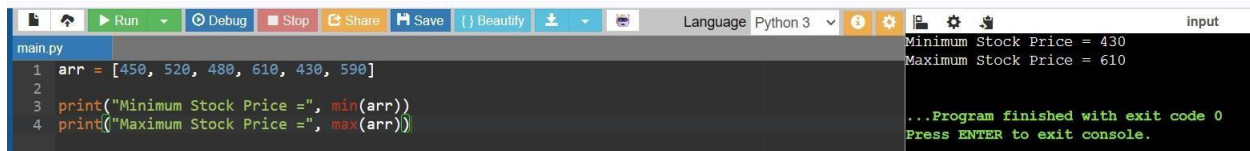
9

14 12 18 11 15 13 17 16 10

Expected Output

Fastest Time = 10

Slowest Time = 18

A screenshot of a Python IDE interface. The editor window shows a file named 'main.py' with the following code:

```
1 arr = [450, 520, 480, 610, 430, 590]
2
3 print("Minimum Stock Price =", min(arr))
4 print("Maximum Stock Price =", max(arr))
```

The output console on the right shows the results: 'Minimum Stock Price = 430' and 'Maximum Stock Price = 610'. Below the output, it says '...Program finished with exit code 0' and 'Press ENTER to exit console.' The IDE has a top bar with 'Run', 'Debug', 'Stop', 'Share', 'Save', and 'Language Python 3'.

Question 10: Largest and Smallest Building Heights

An urban planner is analyzing building heights in a city block and wants to determine the tallest and shortest buildings.

Input

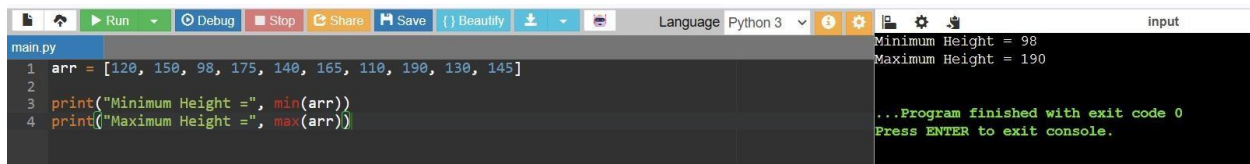
10

120 150 98 175 140 165 110 190 130 145

Expected Output

Minimum Height = 98

Maximum Height = 190

A screenshot of a Python IDE interface. The editor window shows a file named 'main.py' with the following code:

```
1 arr = [120, 150, 98, 175, 140, 165, 110, 190, 130, 145]
2
3 print("Minimum Height =", min(arr))
4 print("Maximum Height =", max(arr))
```

The output console on the right shows the results: 'Minimum Height = 98' and 'Maximum Height = 190'. Below the output, it says '...Program finished with exit code 0' and 'Press ENTER to exit console.' The IDE has a top bar with 'Run', 'Debug', 'Stop', 'Share', 'Save', and 'Language Python 3'.